

Practica 6 Paralela

Rodrigo Pelayo

Diciembre 2025

1. Diseño del Software

Se nos pide crear un programa que ejecute 4 tareas planificadas en 4 hilos diferentes con las siguientes características:

Thread	C	T	D
π_1	0.15	1.4	1.4
π_2	0.6	2.9	2.9
π_3	2.7	13.0	13.0
π_4	5.3	50.0	50.0

Cuadro 1: Tabla de Tareas

Mientras que los hilos van ejecutando sus tareas, el hilo principal ha de imprimir cada 5 segundos los peores tiempos registrados por cada uno de los hilos en realizar su tareas.

La clave de mi implementacion se centra en 3 cosas, la informacion que entrego a cada hilo en forma de estructura, como guardo los mejores tiempos y la funcion que asigno a cada hilo.

La informacion va en una estructura de esta forma:

Listing 1: Estructura de Argumentos

```
struct hilo_args {  
    int idx; // indice del hilo  
    struct timespec wait_time;  
    struct timespec period;  
};
```

Así con esta información podré usar una única función declarada para todos los hilos, `function()`. Esta función usará el campo `idx` de la estructura para saber dónde guardar los tiempos en la variable global `worst_times`, `wait_time` para saber cuánto tiempo pasarle como parámetro a la función `eat` y `period` para saber cada cuánto tiempo llamar a esa función `eat`.

Luego en el main usaré este bloque de codigo para imprimir periódicamente los peores resultados cada 5 segundos:

Listing 2: While

```
while (1){  
    sleep(5); // Espera 5 segundos  
    for (int i = 0; i < NUM_THREADS; i++) {  
        report ("Peor_Tiempo_", i, &worst_times[i]);  
    }  
}
```

2. Desarrollo de las pruebas realizadas

2.1. Actividad 1

En la actividad 1 me salen estos resultados:

Thread	Tiempo en Linux	Tiempo en Marte OS
π_1	0.171	11,897
π_2	0.617	7,992
π_3	2.717	4,082
π_4	5.306	5,612

Cuadro 2: Tabla de Tareas en Linux vs Marte OS

Aquí hacer un breve inciso y es que mi implementación ha sido ligeramente diferente a la de la clase por lo que mi tiempo en Marte OS es diferente, concretamente en lo referente a Marte OS, que me debería de dar entorno 7-8s. Aun así me sigue sirviendo para explicar los mismos problemas. Como podemos ver en linux sin manejar prioridades me tarda muy parecido al tiempo que estimamos teóricamente a la tarea, sin embargo en Marte OS esto cambia, y pareciera verse inverso al que hayamos puesto en C.

Esto puede explicarse a un manejo diferente de prioridades por defecto del sistema operativo y la composición del procesador por menos unidades físicas.

2.2. Actividad 2

Thread	Tiempo en Linux	Tiempo en Linux con Taskset
π_1	0.171	0,598
π_2	0.617	2,086
π_3	2.717	8,088
π_4	5.306	11,904

Cuadro 3: Tabla de Tareas en Linux vs Linux con taskset

Taskset fija la afinidad de CPU del proceso, es decir, fija que solo use x hilos lógicos concretos, esto hace que todo el programa se vea forzado a usar únicamente un hilo y este se tenga que turnar entre todas las tareas.

2.3. Actividad 3

Thread	Tiempo en Marte OS con prioridades	Tiempo en Marte OS
π_1	0,163	11,897
π_2	0.768	7,992
π_3	4,182	4,082
π_4	12,456	5,612

Cuadro 4: Tabla de Tareas en Marte OS con manejo de prioridades vs Marte OS base

Aquí se puede ver que el primer y segundo hilo tardan relativamente parecido a lo que deberían, sobre todo el hilo 1, pero en cambio el hilo 3 y 4 ya tardan más, esto se debe al manejo de prioridades que hemos puesto, que prima las tareas con menor periodo y menor peor tiempo. Por otro lado, los hilos 3 y 4 ya deben de tardar más porque no hay suficientes cores físicos en el CPU para calcular todo de forma paralela y al tener menor prioridad son constantemente interrumpidos por las otras tareas.

2.4. Actividad 4

Incrementos de 0,2 y 0,3 en Linux con taskset y en Marte OS

Thread	Linux base +0,2	Linux con taskset +0,2	Marte OS +0,2 con prioridades
π_1	0,354	6,022	0,363
π_2	0,810	6,762	1,168
π_3	2,924	11,982	6,707
π_4	5,525	22,066	20,092

Cuadro 5: Tabla de Marte OS con taras/ Tareas Linux + taskset +0,2

Thread	Linux base +0,3	Linux con taskset +0,3	Marte OS +0,3 con prioridades
π_1	0,457	1,394	0,463
π_2	0,918	4,243	1,368
π_3	3,002	11,596	9,022
π_4	5,616	21,333	42.772

Cuadro 6: Tabla de Marte OS con taras/ Tareas Linux + taskset +0,3

Aqui podemos un resultado inesperado y es que con taskset tarda menos con 0,3 mas que con 0,2, esto puede deberse a que el planificador del SO por su cuenta ante la sobrecarga del sistema deja de priorizar sobre fairness y comienza sobre throughput .

Luego, el otro resultado recalable es que el hilo 4 de Marte Os pasa de tardar 20,92 s a 42,092 de +0,2 a +0,3s lo cual es mas del doble esto ejemplifica la sobrecarga de las pocas unidades fisicas.